Apprivoiser le renard

Regroupement de **quatre challenges** d'introduction au web, de difficultés croissantes, pour apprendre à se servir des **DevTools de Firefox**.

1. La Machine à Cookies
2. La Transformation des Cookies
3. Le Maître des Cookies
4. Livraison de Cookies

Flag : `Star{L3s5enT1el_3st_pr35Que_1NvisIb1e_P0ur_l3s_YEuX_!}`

La Machine à Cookies

- **Niveau** : facile
- **Flag** : `Star{L3s5enT1el_`
- **Consigne** : *Grâce à l'inspecteur de Firefox, trouve un moyen de voir la machine à cookies cachée derrière le panneau, et trouve un moyen de cuire 4 cookies.*

Énoncé

Solution

Ce challenge permet de découvrir **l'inspecteur**. Le concept est simple, il suffit d'ouvrir l'inspecteur pour supprimer une div qui en cache d'autres. On découvre alors un formulaire de commande de cookies qui semble bloqué dans l'interface web.

```
▼ <div class="challenge">  flex
  ▼ <div class="machine">  flex
    ▼ <div class="form">  flex
        <label for="cookie-number">Combien de cookies souhaitez-vous cuire ?</label>
        <input id="cookie-number" type="number" name="cookie-number" value="0" disabled="">
        <input id="submit-bake" type="submit" value="Cuire">  event
      </div>
      <img src="../static/img/oven.png" alt="Oven">
    </div>
  ▶ <div id="forbidden-sign" class="sign">…</div>  flex
  </div>
```

Deux possibilités : on enlève le `disabled` pour pouvoir sélectionner dans l'interface web les 4 cookies souhaités, ou on change carrément l'attribut `value` de l'input dans l'inspecteur.

Une fois le formulaire validé, on obtient le premier morceau de flag.

La Transformation des Cookies

- **Niveau** : moyen
- **Flag** : `3st_pr35Que_`
- **Consigne** : *Grâce au debugger de Firefox, trouve la fonction qui transforme une liste d'ingrédients en cookies, trouve un moyen de l'exécuter et trouve un moyen de récupérer le cookie pendant sa conception avant qu'il ne soit détruit.*

Énoncé

Solution

Ce challenge permet de découvrir **le debugger**. S'apparentant à un challenge de Reverse, il est question de trouver deux choses dans le Javascript du site :

- Une fonction qui prépare des cookies
- La liste des ingrédients qu'elle prend en argument

La liste des ingrédients se trouve dans une balise inline `<script>` et se présente comme suit :

```
var liste_des_ingredients = [
  "3reP4ehY89",
  "5gFdes3_i$Ll+",
  "56B4rstt5wx",
  "yzABeçàQG?",
  "I_QLM",
  "Sucres!",
  "OuiXpu@#?!",
  "àéösale",
];
```

La fonction se nomme `bakeCookies`, est trouvable dans `static/script/cookiesTransform.js` et se présente comme suit :

```
const bakeCookies = (ingredients) => {
  let cookie;
  cookie = mix(ingredients);
  cookie = shake(cookie);
  cookie = addSugar(cookie);
  cookie = heat(cookie);
  cookie = present(cookie);
  return cookie;
};
```

On peut donc appeler dans la **console** la fonction avec la liste en arguments pour obtenir :

`bakeCookies(liste_des_ingredients)`

> "Voici le cookie-flag : Ha ha, tout a brûlé au four. Tu n'auras rien méchant Renard !"

Il faut donc réussir à récupérer le cookie avec qu'il ne brûle. En analysant la fonction `bakeCookies` on remarque l'appel à la fonction `heat`. On peut la retrouver dans le fichier :

```
const heat = (prep) => {
  let isRenard = true;
  let heated = "";
  for (let index = 0; index < prep.length; index = index + 3) {
    heated += prep[index];
  }

  heated = heated.slice(0, 12);

  if (isRenard) {
    heated = "Ha ha, tout a brûlé au four. Tu n'auras rien méchant Renard !";
  }

  return heated;
};
```

On remarque que cette fonction renvoie la variable `heated` qui est forcément `"Ha ha, tout a brûlé au four. Tu n'auras rien méchant Renard !"` car la variable `isRenard` est toujours égale à `true.`

On place donc un **breakpoint** avant la ligne :

```
if (isRenard) {
```

et on remarque que la variable heated ressemble à notre suite de flag.

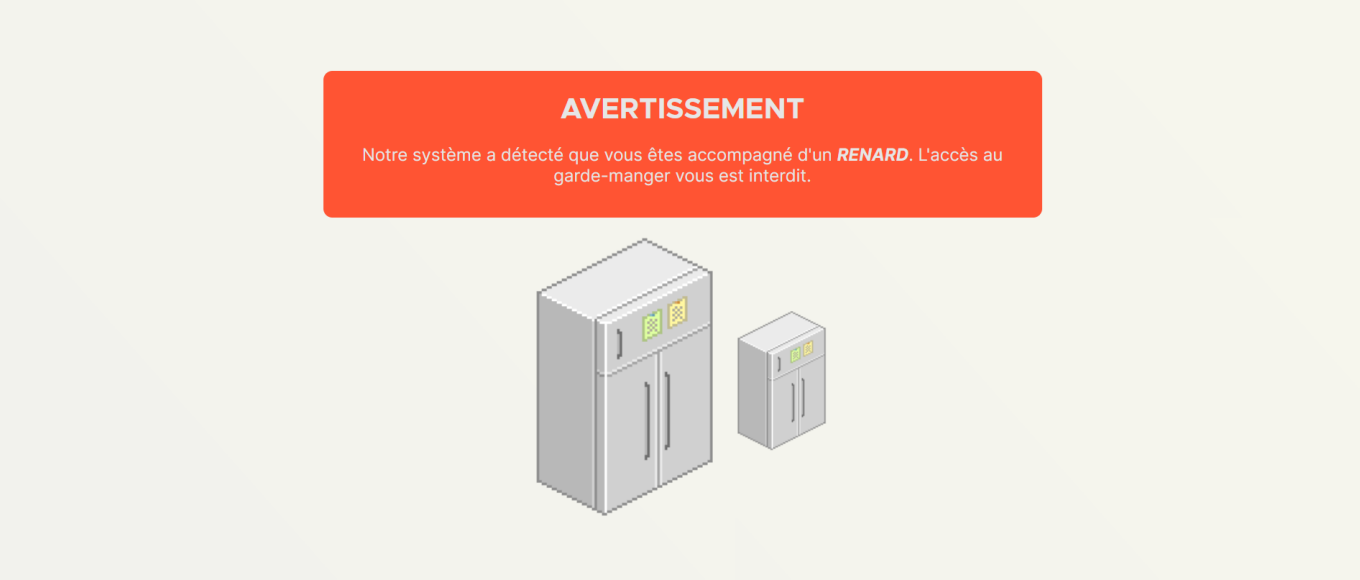
```
48 const heat = (prep) => {
49   let isRenard = true;  isRenard: true
50   let heated = "";
51   for (let index = 0; index < prep.length; index = index + 3) {
52     heated += prep[index];  prep: "3e5sLBtzà_u!p!arhg3l45AQQcOuàleYF_+rwBGLru@éeP8di5sxe?Mei#ö49e$6tyçISsX?s"
53   }
54
55   heated = heated.slice(0, 12);  heated: "3st_pr35Que_"
56
57   if (isRenard) {
58     heated = "Ha ha, tout a brûlé au four. Tu n'auras rien méchant Renard !";
59   }
60
61   return heated;
62 };
```

```
bakeCookies
(global)
Portées
heat
  <this>: Window
  arguments: Arguments
  heated: "3st_pr35Que_"
  isRenard: true
  prep:"3e5sLBtzà_u!p!arhg3l45A
Bloc
Window: Global
Points d'arrêt XHR
```

Le Maître des Cookies

- **Niveau** : facile
- **Flag** : `1NvisIb1e_P0ur_`
- **Consigne** : *Trouve un moyen d'accéder au garde-manger pour faire une surprise à Renard à son réveil !*

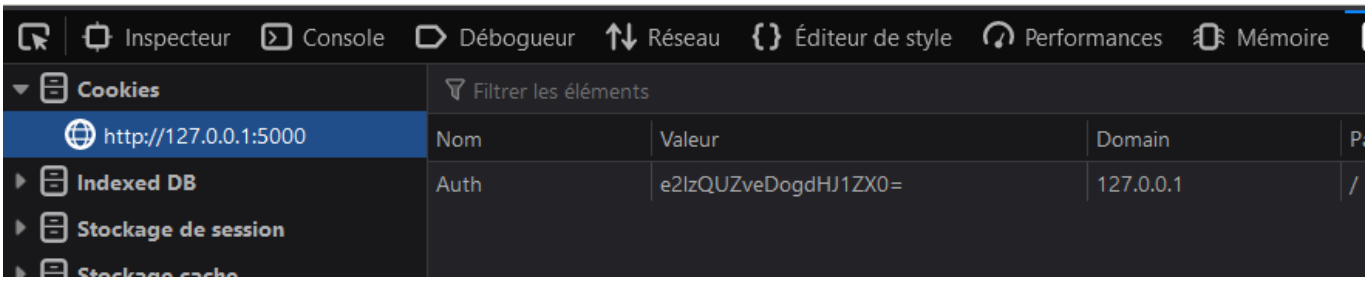
Énoncé



Solution

Ce challenge permet de découvrir **le stockage**. De façon assez classique, il s'agit de trouver le cookie d'*authentification* et de le modifier correctement.

Dans les DevTools, on peut trouver un cookie étrange nommé `Auth` de valeur `e2lzQUZveDogdHJ1ZX0=`.



On reconnaît de la **base64** que l'on va décoder avec un outil comme CyberChef. On trouve alors :

```
{
  isAFox: true;
}
```

Comme nous ne voulons pas être un renard, il suffit alors de changer cela en :

```
{
  isAFox: false;
}
```

que l'on encode en `base64` à nouveau : `e2lzQUZveDogZmFsc2V9`.

Il suffit de coller cette valeur à la place de l'ancienne dans les DevTools et de rafraîchir la page et paf ! Le flag !

Livraison de Cookies

- **Niveau** : moyen
- **Flag** : `l3s_YEuX_!}`
- **Consigne** : *Trouve un moyen de te faire livrer des cookies via le formulaire de commande ci-dessous.*

Énoncé

Solution

Ce challenge permet de découvrir **les requêtes HTTP**. On troue un bon de commande pour cookies. On peut tenter une commande de façon naïve mais on obtient l'erreur suivante :

`Votre commande n'est pas passée par la post et n'a donc pas été traitée.`

Effectivement, si on relit les avertissements on a quelques consignes :

- *Par mesure de sécurité, toutes les commandes doivent passer par la **post** sous risque d'être refusée.*
- *Seules les commandes effectuées depuis des ordinateurs **internes au réseau** de l'entreprise seront prises en compte.*

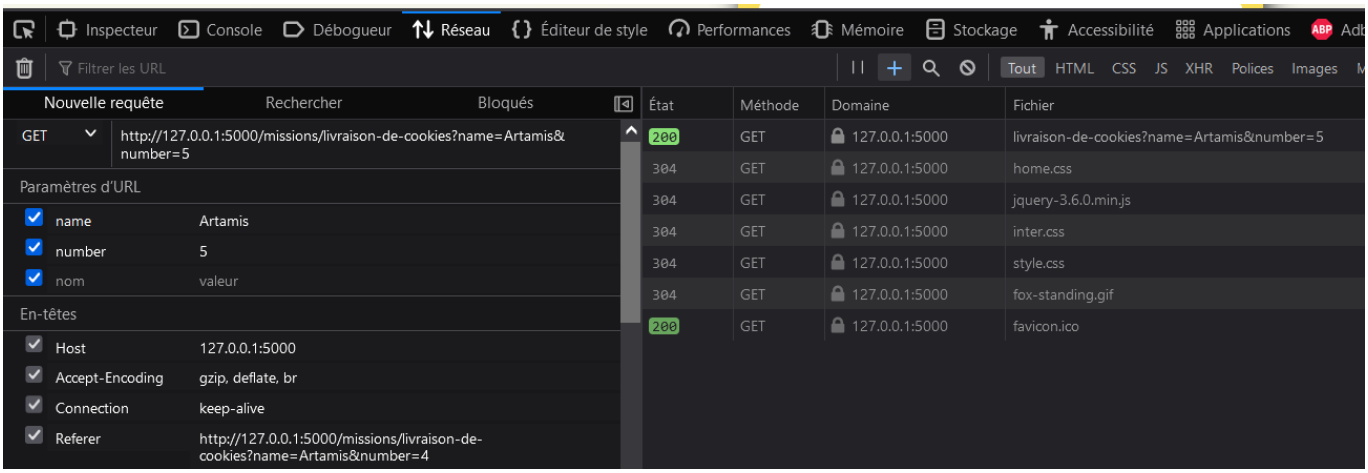
La première est assez claire : on veut envoyer une requête de type `POST`.

La seconde est un peu plus mystérieuse. Après quelques recherchers, on découvre le concept d'IP privées (voir ici par exemple). Il faudrait donc que le server croie que nous sommes bien un ordinateur interne, avec une IP Privée.

Pour cela, un header HTTP peut nous aider : le X-Forwarded-For qui permet d'identifier l'adresse IP d'un client qui se connecterait à un web serveur via un proxy par exemple.

Maintenant, mettons cela en pratique. On ouvre les DevTools à l'onglet **Network** et on envoie un formulaire bateau pour recupérer la requête.

Clic droit dessus puis `Renvoyer`.



Là on remplace le `GET` par `POST` et on ajoute une nouvelle ligne de header :

```
X-Forwarded-For: 192.168.20.21
```

> Rq: N'importe autre adresse IP privée fonctionne

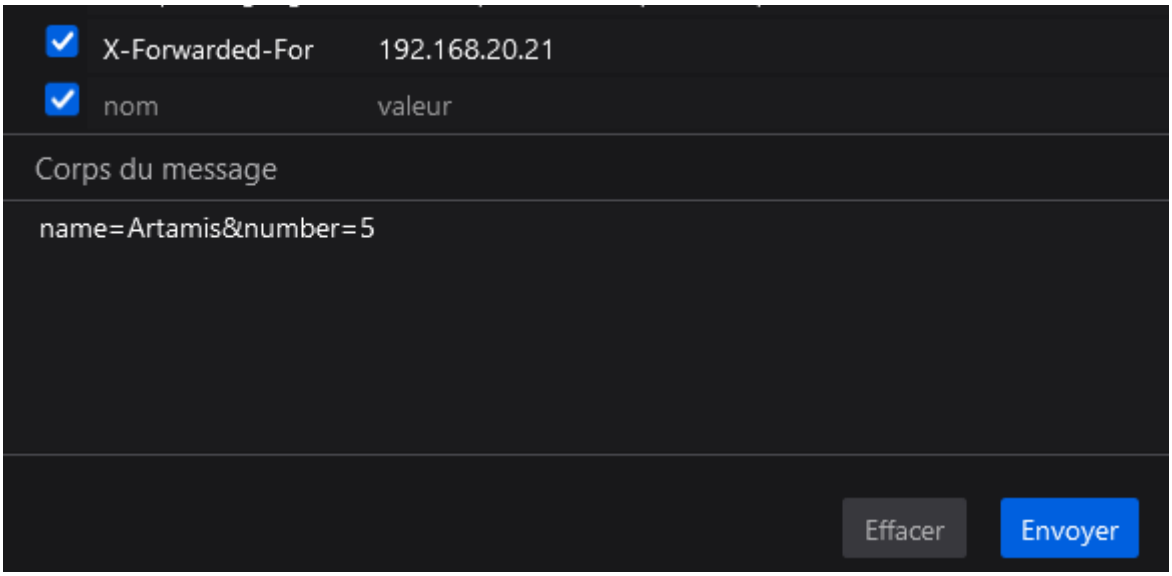
Enfin, on clique sur `Envoyer`.

On va consulter la réponse toujours dans les DevTools et là... erreur : `Votre nom ou le nombre de cookies est manquant ou erroné.`

Eh oui ! Les données d'un formulaire `GET` et d'un formulaire `POST` ne sont pas envoyées de la même façon. Pour un `GET`, les paramètres apparaissent dans l'UTL directement :

```
http://127.0.0.1:5000/missions/livraison-de-cookies?name=Artamis&number=5
```

Mais pour un `POST`, ils doivent apparaître dans le corps de la requête :



On renvoie et... même erreur.

Renseignons-nous plus sur les requêtes `POST` (ici par exemple).

On voit alors une ligne fondamentale : `Le type du corps de la requête est indiqué par l'en-tête Content-Type.`

Il nous faut rajouter un header HTTP :

```
Content-Type: application/x-www-form-urlencoded
```

Et là c'est tout bon ! On obtient un gentil message :

```
La livraison est prévue dans moins de quinze minutes. Veuiller montrer ce flag au
livreur : l3s_YEuX_!}
```

> Rq: Il est possible de faire plus simple. Pour formuler correctement la requête POST, on peut utiliser l'inspecteur pour changer simplement l'attribut `method="get"` en `method=post` dans la balise `<form>` du code HTML. De cette façon la requête sera déjà bien formulée avec les paramètres dans le corps et l'entête `Content-Type`. Il ne reste plus qu'à ajouter le `X-Forwarded-For` et le tour est joué.